Mobilizing Bat Literature

using versioned snapshots of the Zotero BatLit Library

Jorrit Poelen (UC Santa Barbara Cheadle Center, Ronin Institute, GloBI) Aja Sherman (Bat Eco-Interactions Project)

2025-08-26/2025-09-17

Guiding Questions

How to manage the BatLit corpus?

How to create versioned snapshots of the BatLit corpus?

How to share versioned snapshots of the BatLit corpus?

Guiding Questions - Brief Answers

How to manage the BatLit corpus?

We use Zotero to manage our literature corpus.

How to create versioned snapshots of the BatLit corpus?

We use Preston to version our literature corpus [1].

How to share versioned snapshots of the BatLit corpus?

We use Zenodo to allow versioned access to BatLit.

[1] Elliott M.J., Poelen, J.H. & Fortes, J.A.B. (2023) Signing data citations enables data verification and citation persistence. *Sci Data*. https://doi.org/10.1038/s41597-023-02230-y hash://sha256/f849c870565f608899f183ca261365dce9c9f1c5441b1c779e0db49df9c2a19d

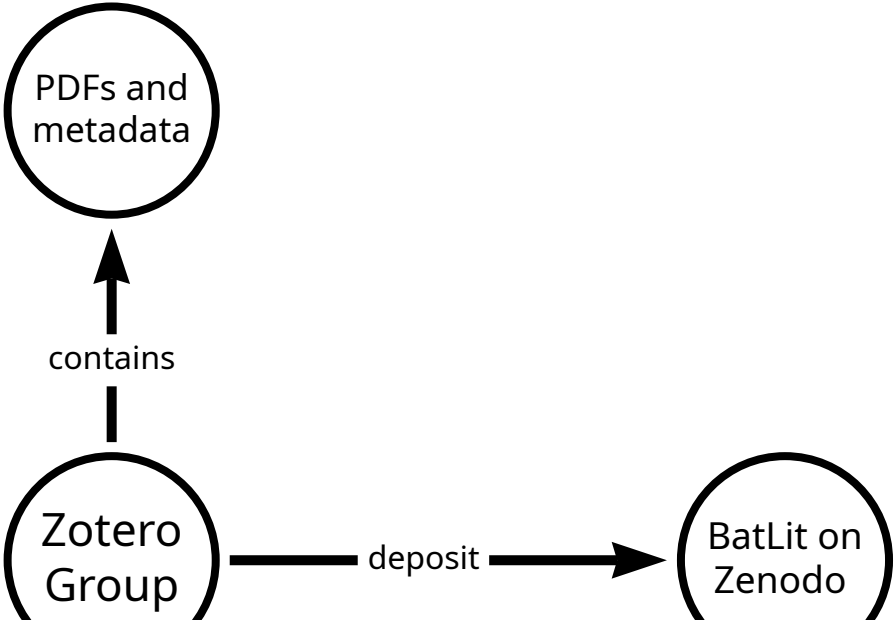
Guiding Questions - More Complete Answers

The BatLit Data Paper describes our BatLit workflow and provides specific examples.

Also, the paper relies on Zotero and Zenodo documentation to answer any questions about these platforms.

The following sections help you get started on Preston and their relation to Zotero and Zenodo.

BatLit Publication Workflow



Why Preston?

Preston allows for creating a versioned snapshot of a Zotero library (or group).

Also, Preston allows for depositing this versioned snapshot into Zenodo.

At time of writing, 26 Aug 2025, this publish Zotero-to-Zenodo functionality is only offered through Preston.

Part I - Preston Basics

1. Tracks (or versions) *Digital* Content
2. Format Agnostic (*any* Digital Content)
3. Allows Signed Data Citations

Prequisites

1. Familiarity with Unix Shell
2. Access to a computer with some Unix Shell
3. Install Preston [2]

[2] https://globalbioticinteractions.org/preston

Check Preston Version

run

```
preston --version
```

and verify that this produces something like:

```
0.11.0
```

Say Hi and Version It

Copy and paste this into your commandline.

```
mkdir -p some/empty-directory
cd some/empty-directory
echo hi there! | preston track --algo md5 | grep hasVersion | preston cat
```

which should produce...

```
hey there!
```

Part I.1 Say Hi and Version It - Step By Step

```
echo hi there!
```

Prints “hi there!” to standard output

Part I.2 Say Hi and Version It - Step By Step

```
echo hi there! | preston track --algo md5
```

Print "hi there!" to output (stdout), then turn this output into input (stdin) of `preston track`. So, this sends "hi there!" to preston and versions (or tracks) the input. The output of preston is a machine readable description of what happened and ends with something like:

```
<...> <...hasVersion> <hash://md5/75c7e31591354f2c82226aa3eb0267c7> <...>
```

This preston output, or description, is formatted in rdf/nquads and records what content was recorded when and by who. This machine readable description is also known as the Bill of Materials (BOM), manifest, or packing slip, for the tracked content.

Part I.3 - Say Hi and Version It - Step By Step

```
echo hi there! | preston track --algo md5 | grep hasVersion
```

This prints only the part of the BOM that includes “hasVersion” and should look something like:

```
<urn:uuid:X> <...hasVersion> <hash://md5/75c7e31591354f2c82226aa3eb0267c7>
```

Step I.4 - Say Hi and Version It - Step By Step

```
<urn:uuid:X> <...hasVersion> <hash://md5/75c7e31591354f2c82226aa3eb0267c7>
```

This is a statement expressed in rdf/nquad. In this case, it expressed something like: there's this thing `urn:uuid:X` that is associated with content that has a cryptographic hash `hash://md5/75c7e31591354f2c82226aa3eb0267c7`. A cryptographic hash is a unique fingerprint derived from the digital content itself. **If the content and the hash algorithm are the same, the fingerprint is always the same.** This concept is central to internet security as well as things like cryptocurrencies.

Part I.5 - Say Hi and Version It - Step By Step

Now, we ask Preston to print the versioned content by piping the “hasVersion” statement into “preston cat“‘:

```
echo hi there! | preston track --algo md5 | grep hasVersion | preston cat
```

to produce . . .

```
hi there!
```

Part I.6 - Say Hi and Version It - Step By Step

Now that we've versioned saying hi, we can print the content using

```
preston cat hash://md5/75c7e31591354f2c82226aa3eb0267c7
```

to produce ...

```
hi there!
```

This suggests that Preston *thinks* that `hash://md5/75c7e31591354f2c82226aa3eb0267c7` is the cryptographic hash of `hi there!`. And . . .

Part I.7 - Say Hi and Version It - Step By Step

If you know the fingerprint of content (e.g., `hash://md5/75c7e31591354f2c82226aa3eb0267c7`), you can use it to ask for what *exactly* what you want. And, on getting a result, you can *independently* verify that this is the case using some commonly available cryptographic hash calculators like `md5sum` (linux) or `md5` (Mac). These calculators are readily available as they are central to internet security and other core applications.

Part I.7 - Say Hi and Version It Continued...

```
preston cat hash://md5/75c7e31591354f2c82226aa3eb0267c7 | md5sum
```

produces:

```
75c7e31591354f2c82226aa3eb0267c7  -
```

Showing that an independent tool `md5sum` verified that the content you asked for is the content you got!

Part I - Takeaways

- Preston tracks, versions and packages digital content
- Cryptographic hashes are unique digital fingerprints for digital content.
- Cryptographic hashes can be generated independently using commonly available tools.
- Cryptographic hashes enable secure citation of digital content

Up Next

- Review BatLit Datapaper
- Create a Zotero Group for Testing
- Track the Zotero Test Group using Preston
- Create a Zenodo Test Community on Zenodo Sandbox
- Publish the Zotero Test Group to the Zenodo Test Community
- Once you feel comfortable, repeat the process with the "real" Zotero BatLit Group and associated Zenodo BatLit Community

Part II - Creating a Snapshot of a Zotero Literature Group

After covering the Preston basics, we now explore how to create a versioned snapshot of a Zotero Literature Collection.

First, we create a Zotero Group for Testing, then we track the Zotero Test Group using Preston

Step II.1- Create A Zotero Group for Testing

1. go to https://zotero.org
2. login using your credentials
3. create a new empty private Zotero group for testing
4. pick two pdfs
5. import the two pdfs into your Zotero Test Group

Part II.2 - Get your Zotero API Key

In order to talk to Zotero using Preston (or any programmatic method) you need a Zotero Web API Key.

1. search Zotero documentation on getting an API Key

   At time of writing, 2025-08-28, the following web pages

   https://www.zotero.org/settings/keys

   https://www.zotero.org/settings/security#applications

2. Get the API key and record it in a safe location. An API key is a combation of letters and number e.g., 12345678

Step II.3 - Take a snapshot of your Zotero Collection Using Preston

Preston has built in functionality to take a snapshot of a Zotero Collection.

1. open a command-line terminal
2. go to a newly created new directory (e.g., `mkdir batlit-test; cd batlit-test`)
3. track the Zotero group by running the following command:

```
export ZOTERO_TOKEN=[SECRET]
preston track --algo md5 [Zotero Group URL]
```

(Tokens are a sequence of numbers and letters, e.g. `ziAJAGw4t5sAaP9APRvBYjkX`)
(Zotero Group URL e.g., https://www.zotero.org/groups/6217595 :warning: make sure to use the numeric group id and *not* the group alias - so, instead of `https://www.zotero.org/groups/bat_literature_project` use `https://www.zotero.org/groups/5435545` :warning:)

Part II.3.1 A sorted list of metadata for most recent Bill of Materials

Create a sorted list metadata statement from the Zotero group for the *most recent* Bill of Materials, and list their content

```
preston head --algo md5\
 | preston cat\
 | grep hasVersion\
 | grep "https://api.zotero.org/groups/[Zotero group number]/items/"\
 | grep -v "file/view"\
 | sort\
 | preston cat\
 > metadata-before-change.txt
```

(replace `[Zotero group number]` with the number found in URL, make sure to remove brackets, e.g., https://www.zotero.org/groups/**6217595**)

Part II.4 Make a change and create a new snapshot

1. Change the title of one of the publication in your test Zotero Group
2. Make a new snapshot version by re-running:

```
preston track --algo md5 [Zotero Group from URL]
```

Part II.4.1 A sorted list of metadata for newly updated Bill of Materials

Create a sorted list metadata statement from the Zotero group for the newly updated Bill of Materials, and list their content

```
preston head --algo md5\
 | preston cat\
 | grep hasVersion\
 | grep "https://api.zotero.org/groups/[Zotero group number]/items/"\
 | grep -v "file/view"\
 | sort\
 | preston cat\
 > metadata-after-change.txt
```

Part II.5 Compare changes in metadata across snapshot versions

After making a change in a Zotero records, and creating a new snapshot, we can compare the different versions of Bill of Materials associated with these snapshots.

In order to do so, we compare (II.3.1) the sorted list of all metadata for associated with an initial Bill of Materials and (II.4.1) a sorted list of all metadata associated with a Bill of Materials after a change.

Next, (II.5.1) we compare the differences between these metadata snapshots.

Part II.5.1 Compare changes across metadata associated with two versions of Bill of Materials

Now that we have the Zotero metadata for the most recent Bill of Materials, as well as a previous version, we can use diff to compare the differences.

```
diff metadata-before-change.txt metadata-after-change.txt
```

to produce a result like

```
[...]
119c119
<             "title": "Pteropus test",
---
>             "title": "Seasonal roosts and foraging movements of the blac
[...]
```

In this example the title was changed to `Pteropus test` and the diff reflects this.

Part II - Next Steps

- Create a Zenodo Test Community on Zenodo Sandbox
- Publish the Zotero Test Group to the Zenodo Test Community

Part III - Deposit a Zotero Snapshot into Zenodo

Now that we can make a snapshot of a Zotera group, we'd like to take this snapshot and deposit it into Zenodo. In order to do so, we need to (III.1) create a test Zenodo community in their sandbox (III.2) create a Zenodo API key, and (III.3) upload a snapshot version into Zenodo sandbox using Preston.

Part III.1 - Create a test Zenodo community

Zenodo provides a sandbox to try their platform and experiment.

1. go to https://sandbox.zenodo.org
2. login
3. create a Zenodo community. Note a community has a name and a id. Note that ids are lowercase only (e.g., bat-lit-2025-05-05).

Part III.2 - Generate A Zenodo API Token

To deposit content into Zenodo programmatically (e.g., using Preston), you need a Zenodo Web API Key / Token.

1. go to https://sandbox.zenodo.org
2. select account > applications > personal access token (?) with all options (e.g., write access, publish) enabled
3. generate a new token (or api key) and record this somewhere safe

Part III.3 - Generate Zenodo Metadata for a Zotero Snapshot

1. open a command-line terminal
2. generate Zenodo metadata for a specific snapshot version

```
preston head --algo md5\
 | preston cat\
 | preston zotero-stream --algo md5 --community "[your community id]"\
 > zenodo.json
```

Note: you can find the community id by looking at the browser address bar. E.g., with https://sandbox.zenodo.org/communities/batlit-test-2025-05-05/settings, "batlit-test-2025-05-05" is the community id.

Part III.4 - Deposit Zenodo Records

1. open a command-line terminal
2. set the evironment variable for Zenodo API access

```
export ZENODO_ENDPOINT=https://sandbox.zenodo.org
export ZENODO_TOKEN=[your secret token]
```

3. deposit Zenodo records using the Zenodo metadata

```
mkdir -p logs
cat zenodo.json\
 | preston track --algo md5 --data-dir logs/data\
 | preston zenodo --algo md5 --data-dir logs/data \
 --remote file://$PWD/data/ \
 --community "[your community id]"\
 > deposit.log
```

Part IV - Maintenance

After initial deposit, metadata in Zotero records may be updated. Also, PDFs associated with a Zotero record may be exchanged with a different one (e.g., an incorrect pdf was associated with some literature record).

To update metadata of a Zenodo deposit associated with a Zotero record, you can re-run Step III.4 after including either `--update-metadata-only` or `--new-version`.

With `--update-metadata-only` an metadata of an existing Zenodo record is updated with the Zotero record metadata. No new version is created and the pdf attachment is left untouched.

With `--new-version` a new version of an existing Zenodo record is deposited with updated the Zotero record metadata and the associated pdf in Zotero.

Note that Zenodo record metadata is editable, however Zenodo record files are *not*.

So, when you need to update a pdf associated with a Zotero record, you need to create a `--new-version`. This is *not* done by default.

Default behavior is to not update the associated Zenodo record and skip the deposit.

Part IV.1 - Edit Existing Record and Update Metadata

1. Update metadata for an already deposited Zotero record.
2. Re-run the III.4 deposit workflow with the `--update-metadata-only`

```
[...]
 | preston zenodo --update-metadata-only --algo md5 [...]
```

3. Verify that the metadata of record in Zenodo was updated, but no new version was created.

Part IV.2 - Create New Record with Updated PDF and Metadata

1. Replace a pdf attachment for an already deposited Zotero record.
2. Re-run the III.4 deposit workflow with the `--new-version`

```
[...]
 | preston zenodo --new-version --algo md5 [...]
```

3. Verify that a new version was created for the Zenodo record including the updated pdf

Part IV.3 - Retire Zenodo Deposit Associated with Deleted Zotero Record

(for now, manual workflow, can be automated if needed)

1. Locate the Zenodo Deposit Associated with a Zotero record that no longer exists
2. Click on Community :gear: icon “submit to community” in lower right panel
3. Submit to the “batlit-retired” community
4. Click on “manage communities”
5. If present, remove the deposit from the BatLit and BLR communities

Part V.1 - BatLit Release Introduction.

Metadata for BatLit releases are in the data folder of https://github.com/bat-literature/bat-literature.github.io . A new release needs to be linked to the older releases to enable version tracking between versions. This is why we need to clone or update the github repository first before making a snapshot of the BatLit Zotero group. Then, after making the snapshot, we include the tracked metadata and commit this to the repository. Also, we keep a copy of the metadata + pdfs elsewhere. So, you need to backup the data folder including metadata and pdfs.

Part V.2 - Clone/Update BatLit Repository

Make sure to install `git` on your system by running:

```
sudo apt install git
```

check whether git installed by

```
git --version
```

this should produce something like:

```
git version 2.43.0
```

then, run

```
git clone https://github.com/bat-literature/bat-literature.github.io
```

to "clone" (or create a copy of) the BatLit repository. By default, the repository is cloned into a folder with the same name as the repository (e.g., `bat-literature.github.io`).

Part V.3 - Verify Current BatLit Version

1. go into `bat-literature.github.io` directory using `cd`
2. run `preston head --algo md5`
3. Compare the output of the command with the existing version on https://batlit.org/datapaper

Part V.4 - Create a Zotero Snapshot

1. Set Zotero API Key with read-only access to the BatLit Zotero
2. Go into the `bat-literature.github.io` directory
3. Verify that a preston history exists by running `preston history --algo md5`
4. Create a snapshot of the BatLit Zotero group

Part VI.1- Update the "HEAD" or most recent version

1. open a terminal
2. go into the `bat-literature.github.io` directory
3. make sure that the `HEAD` file contain the most recent hash of the BatLit version by running

```
preston head --algo md5 > HEAD
```

and verifying that the first hash is the same as the one in the `HEAD` file by running

```
cat HEAD
```

Part VI.2- Update the Reference Lists for Website/search

1. open a terminal
2. go into the `bat-literature.github.io` directory
3. verify that the program "miller" is installed by running

```
mlr --version
```

If not install, run:

```
sudo apt install miller
```

4. to generate a table of the most recent bat lit references, run the command:

```
bin/list-refs.sh\
 | tee zotero/refs.csv
```

5. inspect the file `zotero/refs.csv`
6. Now, generate the files `zotero/refs.tsv`, `zotero/refs-100.csv` and `zotero/refs-100.tsv`

```
using cat zotero/refs.csv\   | mlr --icsv --otsvlite cat\    >
```

Part VI.3- Link BatLit references to their Zenodo deposits

Now that we've created a reference table for the versioned Zotero records in `zotero/refs.csv`, we'd like to add the Zenodo deposits associated with the Zotero records.

For instance, the Zotero record identified by `urn:lsid:zotero.org:groups:5435545:items:NBLJKXN9` is associated a reference with title "Lagos Bat Virus, an Under-Reported Rabies-Related Lyssavirus". This reference is included in some specific row in the file `zotero/refs.csv`.

Following, we'd like to link this Zotero records to their counterpart in Zenodo. We do this via the identifier `urn:lsid:zotero.org:groups:5435545:items:NBLJKXN9` . This identifier is embedded in the Zenodo deposits, and can be used to locate them in Zenodo.

Finding a record associated with `urn:lsid:zotero.org:groups:5435545:items:NBLJKXN9` can be done using:

https://zenodo.org/search?q=%22urn%3Alsid%3Azotero.org%3Agroups%3A5435545%3Aitems